R3.07
SQL dans un langage de programmation

Chapitre 7

Programmer dans la base : procédures, exceptions, curseurs, triggers



Selena Lamari et Chouki Tibermacine

BUT 2 Informatique — IUT de Vannes — Université Bretagne Sud

Avant de commencer

La syntaxe de base est dans le TP6

Le **mini-tutoriel PL/pgSQL** du sujet de TP6 présente la syntaxe : structure d'une procédure, `DECLARE`, `SELECT ... INTO`, `IF`, `RAISE EXCEPTION`, curseur simple, fonction de trigger, `OLD`/`NEW`, `CREATE TRIGGER`. Le sujet liste aussi des exemples de procédures et de triggers pour l'application bancaire.

Ce chapitre ne la répète pas. Il répond aux questions que le tutoriel laisse ouvertes :

- **faut-il** mettre cette logique dans la base ?
- fonction ou procédure ? peut-on valider (`COMMIT`) dans une procédure ?
- comment **rattraper** une erreur en PL/pgSQL, et comment Java la reçoit-il ?
- quand un curseur est-il utile, côté base et côté JDBC ?
- quels triggers au-delà du `FOR EACH ROW` simple, et quels pièges ?
- comment sécuriser et versionner ce code ?

Plan du cours

1. Où placer la logique ?
2. Fonctions et procédures
3. Exceptions
4. Curseurs
5. Triggers : au-delà des bases
6. Sécuriser et industrialiser

Java ou base de données : les critères

Critère	Plutôt dans la base	Plutôt en Java
Clients de la base	plusieurs applications, scripts, accès SQL direct	une seule application
Volume traité	beaucoup de lignes, peu de résultat	peu de lignes
Allers-retours	nombreux (boucle de requêtes)	un ou deux
Tests, débogage	—	outils Java (JUnit, IDE)
Montée en charge	—	on ajoute des serveurs d'application ; la base est plus dure à multiplier
Portabilité	—	PL/pgSQL n'existe que sur PostgreSQL

Le sujet du TP6 liste les avantages des procédures et triggers ; ce tableau rappelle qu'ils ont aussi un prix.

La règle de l'outil le plus simple

1. **Contrainte déclarative** d'abord : `NOT NULL`, `CHECK`, `UNIQUE`, clé étrangère
2. **Trigger** si la règle dépend d'autres lignes ou d'autres tables, ou s'il faut une action automatique (audit)
3. **Procédure** pour une opération métier complète et nommée (virement, clôture)

Exemple : « interdire un solde négatif » (liste d'exemples du TP6)

Un trigger fonctionne, mais une contrainte suffit, est plus rapide et ne peut pas être oubliée :

```
ALTER TABLE Compte ADD CONSTRAINT chk_solde CHECK (solde >= 0);
```

Sa violation renvoie un code précis (23514) et le nom de la contrainte à Java.

Plan du cours

1. Où placer la logique ?
2. Fonctions et procédures
3. Exceptions
4. Curseurs
5. Triggers : au-delà des bases
6. Sécuriser et industrialiser

Fonction ou procédure ?

	`FUNCTION`	`PROCEDURE` (PostgreSQL 11+)
Appel	dans une requête : `SELECT f(...), FROM f(...), WHERE`	`CALL p(...)` uniquement
Résultat	une valeur, une ligne ou une table (`RETURNS TABLE`, `SETOF`)	aucun, ou paramètres `OUT`/`INOUT`
`COMMIT`/`ROLLBACK` internes	interdits	autorisés (sous conditions)
Usage typique	calcul, requête paramétrée, fonction de trigger	opération métier, traitement par lots

- Le relevé de compte du TP6 est une **fonction** (on l'interroge avec `SELECT`) ; le virement est une **procédure**

Valider par lots dans une procédure

```
CREATE PROCEDURE verser_interets(
    p_taux NUMERIC, p_lot INT DEFAULT 1000)
LANGUAGE plpgsql AS $$
DECLARE c RECORD; n INT := 0;
BEGIN
  FOR c IN SELECT numCompte FROM Compte
           WHERE typeCompte = 'EPARGNE' LOOP
    UPDATE Compte
       SET solde = round(solde * (1 + p_taux), 2)
     WHERE numCompte = c.numCompte;
    n := n + 1;
    IF n % p_lot = 0 THEN
      COMMIT;                    -- lot valide
    END IF;
  END LOOP;
END $$;
```

- Traitement *batch* (le PN cite les « batch et procédures stockées »)
- Un incident en cours de route ne fait perdre que le lot courant
- Les verrous sont relâchés à chaque `COMMIT`

Interdit (testé) :

- `COMMIT` dans une fonction
- `CALL` dans un `BEGIN` explicite

```
ERROR: invalid transaction termination
```

Conséquence côté Java

- En `autoCommit = false`, le pilote JDBC ouvre lui-même une transaction avant chaque requête
- Une procédure qui fait `COMMIT` doit donc être appelée en **autocommit**

```
c.setAutoCommit(true);
try (Statement s = c.createStatement()) {
  s.execute("CALL verser_interets(0.02)");              // OK
}
c.setAutoCommit(false);
try (Statement s = c.createStatement()) {
  s.execute("CALL verser_interets(0.02)");              // SQLState 2D000
}                                                       // invalid transaction
    termination
```

- Une procédure **sans** `COMMIT` interne (le virement) s'appelle normalement dans la transaction de l'application

Volatilité d'une fonction

- `VOLATILE` (défaut) : résultat imprévisible, réévaluée à chaque ligne (`random()`, `now()` entre deux transactions, tout ce qui écrit)
- `STABLE` : même résultat pendant une requête, peut lire la base
- `IMMUTABLE` : ne dépend que de ses arguments

Lien avec le chapitre 3 : un **index sur expression** exige une fonction `IMMUTABLE`

```
CREATE FUNCTION categorie(s NUMERIC) RETURNS TEXT LANGUAGE plpgsql
  AS $$ BEGIN RETURN CASE WHEN s > 1000 THEN 'haut' ELSE 'bas' END; END $$;
CREATE INDEX ON Compte (categorie(solde));
```

```
ERROR:  functions in index expression must be marked IMMUTABLE
```

Déclarer la bonne volatilité aide aussi l'optimiseur. Mentir (`IMMUTABLE` sur une fonction qui lit une table) donne des index faux.

Appeler fonctions et procédures depuis Java

```
// Fonction qui renvoie une table : une requete comme une autre
PreparedStatement ps = c.prepareStatement("SELECT * FROM releve_compte(?)");

// Procedure : CALL, en convertissant les types si besoin
PreparedStatement ps = c.prepareStatement("CALL virement(?, ?, CAST(? AS REAL))");
```

- PostgreSQL choisit la procédure d'après les **types** des arguments ; `setDouble` envoie un `double precision` : sans `CAST`, `procedure virement(integer, integer, double precision) does not exist`
- Avec Spring Data : `@Query(value = "CALL ...", nativeQuery = true)` (chapitre 6)
- `@Procedure` (Spring Data) passe les paramètres **par nom** : ils doivent porter le même nom en Java et en SQL, sinon l'appel échoue (testé)

Plan du cours

1. Où placer la logique ?
2. Fonctions et procédures
3. Exceptions
4. Curseurs
5. Triggers : au-delà des bases
6. Sécuriser et industrialiser

Lever une erreur exploitable

```
RAISE EXCEPTION 'Solde insuffisant sur le compte %', p_num
  USING ERRCODE = 'P0002',
        DETAIL  = format('solde = %s, demande = %s', v_solde, p_montant),
        HINT    = 'Reduire le montant';
```

```
ERROR:  Solde insuffisant sur le compte 3
DETAIL:  solde = 10, demande = 50
HINT:  Reduire le montant
```

- Niveaux : `DEBUG`, `LOG`, `INFO`, `NOTICE`, `WARNING` (simple message) ; `EXCEPTION` (arrête et annule)
- `ERRCODE` : un code par erreur métier, pour que Java les distingue sans analyser le texte. Par défaut : `P0001`

Rattraper une erreur : le bloc EXCEPTION

```
BEGIN
  INSERT INTO journal(msg) VALUES ('avant');
  BEGIN                                    -- sous-bloc
    INSERT INTO journal(msg) VALUES ('dans le bloc');
    UPDATE Compte SET solde = -5 WHERE numCompte = 1;
  EXCEPTION
    WHEN check_violation THEN
      INSERT INTO journal(msg)
      VALUES ('rattrape : ' || SQLSTATE);
  END;
END;
```

```
avant
rattrape : 23514
```

- Conditions : nom (`unique_violation`, `check_violation`…), `SQLSTATE '23505'`, ou `OTHERS`
- Tout ce que le sous-bloc a fait est annulé : la ligne « dans le bloc » a disparu
- Le bloc pose un point de sauvegarde : il coûte cher, à éviter dans une boucle sur des milliers de lignes
- Ne pas tout avaler avec `OTHERS` : journaliser puis relancer (`RAISE;`)

Inspecter l’erreur rattrapée

```
EXCEPTION WHEN OTHERS THEN
  GET STACKED DIAGNOSTICS v_etat       = RETURNED_SQLSTATE,
                          v_msg        = MESSAGE_TEXT,
                          v_contrainte = CONSTRAINT_NAME;
  RAISE NOTICE 'etat=% contrainte=% msg=%', v_etat, v_contrainte, v_msg;
  RAISE;                                          -- relance la meme erreur
```

```
NOTICE:  etat=23505 contrainte=compte_pkey msg=duplicate key value violates unique constraint "
     compte_pkey"
```

- Utile pour journaliser dans une table d’erreurs avant de relancer
- Autres champs : `PG_EXCEPTION_DETAIL`, `PG_EXCEPTION_HINT`, `TABLE_NAME`, `COLUMN_NAME`...

L’erreur vue de Java

```java
} catch (PSQLException e) {                          // org.postgresql.util
  ServerErrorMessage m = e.getServerErrorMessage();
  switch (e.getSQLState()) {
    case "P0002" -> throw new SoldeInsuffisantException(m.getDetail());
    case "23505" -> throw new DoublonException(m.getConstraint());
    case "23514" -> throw new RegleMetierException(m.getConstraint());
    default      -> throw new DataAccessException(e);
  }
}
```

23505 unicité	23503 clé étrangère	23514 CHECK	23502 NOT NULL
40001 sérialisation (ch. 2)	40P01 interblocage (ch. 2)	P0001 RAISE par défaut	P0002… vos codes

Testé : `getDetail()` renvoie le `DETAIL` du `RAISE` (« solde = 10 »), `getConstraint()` le nom de la contrainte violée (« chk_solde »).

Plan du cours

1. Où placer la logique ?
2. Fonctions et procédures
3. Exceptions
4. Curseurs
5. Triggers : au-delà des bases
6. Sécuriser et industrialiser

Curseurs côté base : quand ?

- Un curseur traite le résultat **ligne par ligne** sans le charger en entier
- `FOR ligne IN SELECT ... LOOP` ouvre un curseur implicite : c'est la forme à privilégier
- Curseur explicite (`OPEN`/`FETCH`/`CLOSE`, vu dans le tutoriel) quand on veut contrôler le parcours : plusieurs curseurs en parallèle, arrêt anticipé, curseur paramétré (`CURSOR (p INT) FOR ...`)

Mais d'abord : peut-on l'écrire en une requête ?

- Exercice 3 du TP6 : le relevé avec curseur et la version `sum(...) OVER (...)` donnent le même résultat
- Une requête ensembliste laisse l'optimiseur choisir (index, parallélisme) ; une boucle impose l'ordre et le détail
- Curseur justifié : décision différente pour chaque ligne, appel d'autres procédures, validation par lots

Curseurs côté JDBC : lire un gros résultat

- Par défaut, le pilote PostgreSQL **charge tout le résultat en mémoire** avant le premier `next()`
- Avec `setFetchSize(n)`, il utilise un curseur serveur et lit par paquets de *n* lignes… seulement si l'autocommit est désactivé

```
c.setAutoCommit(false);                        // indispensable
try (PreparedStatement ps = c.prepareStatement("SELECT id, txt FROM big")) {
  ps.setFetchSize(1000);
  try (ResultSet rs = ps.executeQuery()) { while (rs.next()) { ... } }
}
c.commit();
```

2 millions de lignes, `fetchSize = 1000`	**1re ligne reçue**	**mémoire**
`autoCommit = true` (fetchSize ignoré)	458 ms	tout le résultat
`autoCommit = false`	1 ms	un paquet

Plan du cours

1. Où placer la logique ?
2. Fonctions et procédures
3. Exceptions
4. Curseurs
5. Triggers : au-delà des bases
6. Sécuriser et industrialiser

Les variantes de triggers

	`BEFORE`	`AFTER`
`FOR EACH ROW`	peut **modifier** `NEW` (valeur calculée) ou renvoyer `NULL` (ligne ignorée)	voit la ligne finale ; valeur de retour ignorée (audit, propagation)
`FOR EACH STATEMENT`	une fois par instruction, avant	une fois par instruction ; accès à toutes les lignes via les **tables de transition**

- Filtres : `UPDATE OF colonne` (seulement si cette colonne est citée), `WHEN (condition)` (évaluée avant d'appeler la fonction)
- Plusieurs triggers sur le même événement : exécutés par **ordre alphabétique** de nom
- Variables disponibles : `TG_OP` (`INSERT`/`UPDATE`/`DELETE`), `TG_TABLE_NAME`, `TG_WHEN`…

BEFORE : compléter la ligne, avec filtres

```
CREATE FUNCTION trg_horodater() RETURNS trigger LANGUAGE plpgsql AS $$
BEGIN
  NEW.maj := now();                    -- on modifie la ligne avant ecriture
  RETURN NEW;
END $$;

CREATE TRIGGER compte_horodater
  BEFORE UPDATE OF solde ON Compte
  FOR EACH ROW
  WHEN (OLD.solde IS DISTINCT FROM NEW.solde)
  EXECUTE FUNCTION trg_horodater();
```

- Testé : `SET solde = solde` (inchangé) et `SET type = type` (autre colonne) n'horodatent pas ; `SET solde = solde + 1` horodate
- `WHEN` évite même l'appel de la fonction : moins coûteux qu'un `IF` à l'intérieur

Un trigger d'audit pour toutes les tables

```
CREATE TABLE audit(id BIGSERIAL, quand TIMESTAMPTZ DEFAULT now(), utilisateur TEXT DEFAULT current_user,
                   table_ TEXT, op TEXT, avant JSONB, apres JSONB);

CREATE FUNCTION trg_audit() RETURNS trigger LANGUAGE plpgsql AS $$
BEGIN
  INSERT INTO audit(table_, op, avant, apres) VALUES (TG_TABLE_NAME, TG_OP,
    CASE WHEN TG_OP <> 'INSERT' THEN to_jsonb(OLD) END,
    CASE WHEN TG_OP <> 'DELETE' THEN to_jsonb(NEW) END);
  RETURN NULL;
END $$;

CREATE TRIGGER compte_audit AFTER INSERT OR UPDATE OR DELETE ON Compte
  FOR EACH ROW EXECUTE FUNCTION trg_audit();
CREATE TRIGGER client_audit AFTER INSERT OR UPDATE OR DELETE ON Client
  FOR EACH ROW EXECUTE FUNCTION trg_audit();       -- meme fonction
```

Une seule fonction, réutilisée par toutes les tables, contre une fonction par table dans l'exercice 2 du TP6.

Garde l'état avant/après, l'auteur et la date (AC24.02 : intégrité et traçabilité).

Niveau instruction et tables de transition

```
CREATE FUNCTION trg_gros_volume() RETURNS trigger LANGUAGE plpgsql AS $$
DECLARE v_nb INT; v_total NUMERIC;
BEGIN
  SELECT count(*), sum(n.solde - o.solde) INTO v_nb, v_total
    FROM nouveaux n JOIN anciens o USING (numCompte);      -- toutes les lignes modifiees
  IF v_nb > 100 THEN
    INSERT INTO alerte(nb, total) VALUES (v_nb, v_total);
  END IF;
  RETURN NULL;
END $$;

CREATE TRIGGER compte_volume AFTER UPDATE ON Compte
  REFERENCING OLD TABLE AS anciens NEW TABLE AS nouveaux
  FOR EACH STATEMENT EXECUTE FUNCTION trg_gros_volume();
```

- Un seul appel pour un `UPDATE` de 10 000 lignes, au lieu de 10 000 avec `FOR EACH ROW`
- Permet des contrôles globaux : volume, total déplacé, cohérence entre lignes

Les pièges des triggers

- **Annulation silencieuse** : un `BEFORE` qui renvoie `NULL` ignore la ligne sans erreur

```
DELETE FROM Compte WHERE numCompte = 9;   -- DELETE 0, aucune erreur pour l'application
```

  Pour refuser une opération : `RAISE EXCEPTION`, que Java recevra
- **Logique cachée** : le développeur Java ne voit pas qu'un `UPDATE` écrit dans trois autres tables ; documenter dans le README
- **Cascades** : un trigger qui modifie une table qui a elle-même des triggers…
- **Coût** : un trigger ligne par ligne s'exécute pour chaque ligne d'un `UPDATE` de masse
- **JPA** : une colonne modifiée par un trigger n'est pas vue par l'entité en mémoire ; relire avec `em.refresh(entite)`, ou déclarer la colonne `@Generated` (Hibernate)

Plan du cours

1. Où placer la logique ?
2. Fonctions et procédures
3. Exceptions
4. Curseurs
5. Triggers : au-delà des bases
6. Sécuriser et industrialiser

N’autoriser que les procédures : SECURITY DEFINER

```
CREATE ROLE appli LOGIN PASSWORD '...';          -- compte utilise par l'application Java
GRANT SELECT ON Compte TO appli;                 -- lecture seule sur la table

CREATE PROCEDURE debiter(p_num INT, p_m NUMERIC)
LANGUAGE plpgsql
SECURITY DEFINER                                 -- s'execute avec les droits du proprietaire
SET search_path = public                         -- evite le detournement par un autre schema
AS $$ BEGIN ... END $$;

REVOKE EXECUTE ON PROCEDURE debiter(INT, NUMERIC) FROM PUBLIC;
GRANT  EXECUTE ON PROCEDURE debiter(INT, NUMERIC) TO appli;
```

```
appli=> UPDATE Compte SET solde = 0 WHERE numCompte = 1;   ERROR: permission denied for table compte
appli=> CALL debiter(1, 10);                               CALL
```

Même si l’application est compromise (injection SQL), elle ne peut modifier les soldes que par les règles de la procédure (AC24.02).

SQL dynamique sans injection

```
CREATE FUNCTION compter(p_table TEXT, p_type TEXT) RETURNS BIGINT LANGUAGE plpgsql AS $$
DECLARE n BIGINT;
BEGIN
  EXECUTE format('SELECT count(*) FROM %I WHERE type = $1', p_table)   -- %I : identifiant
     INTO n USING p_type;                                              -- $1 : valeur
  RETURN n;
END $$;
```

```
SELECT compter('cpt; DROP TABLE cpt', 'x');
ERROR:  relation "cpt; DROP TABLE cpt" does not exist
```

- Jamais de concaténation `'... ' || p_table || ' ...'` : c'est l'injection SQL du chapitre 1, côté serveur
- `%I` pour les noms de tables et colonnes, `USING` pour les valeurs (comme un `PreparedStatement`)

Versionner le code de la base

- Le code PL/pgSQL est du code : il va dans le dépôt Git, avec des scripts numérotés

```
src/main/resources/db/migration/
  V1__schema.sql
  V2__contraintes.sql
  V3__procedure_virement.sql
  V4__audit.sql
```

- Un outil de migration (**Flyway**, Liquibase) applique au démarrage les scripts pas encore passés, dans l'ordre, et garde l'historique dans une table ; avec Spring Boot, il suffit d'ajouter les dépendances `flyway-core` et `flyway-database-postgresql`
- `CREATE OR REPLACE` pour faire évoluer une fonction sans la supprimer
- Tester les procédures : tests d'intégration Java contre une vraie base PostgreSQL (conteneur Docker)

À retenir

- Contrainte d'abord, trigger ensuite, procédure pour une opération métier nommée
- Fonction = interrogeable en SQL, sans `COMMIT` ; procédure = `CALL`, peut valider par lots (en autocommit côté JDBC)
- Erreurs : `RAISE ... USING ERRCODE` côté base, `getSQLState()` côté Java ; un bloc `EXCEPTION` annule son contenu
- Curseurs : préférer une requête ensembliste ; côté JDBC, `setFetchSize` + autocommit désactivé
- Triggers : `BEFORE` pour compléter, `AFTER` pour tracer, niveau instruction pour les gros volumes ; jamais d'annulation silencieuse
- Sécurité : `SECURITY DEFINER` + `GRANT EXECUTE`, `format('%I')` + `USING`

Questions ?



Chouki.Tibermacine@univ-ubs.fr